Dynamic Dependency Learning and Sparse Segment-Based Pattern Extraction for Multivariate Time Series Load Forecasting

1st Nguyen Thi Ngoc Anh 2nd Nguyen Hai Long 3rd Bui Hoang Tien Dat

***Abstract*—Dự báo phụ tải đa biến cần khai thác đồng thời tính lặp lại có chu kỳ của từng chuỗi và sự phụ thuộc giữa các biến vốn thưa và thay đổi theo thời gian, trong khi xu hướng dùng cửa sổ quá khứ dài khiến chi phí tính toán của các mô hình dự báo ngày càng lớn. Bài báo đề xuất DyBAS-Net, mô hình chia mỗi chuỗi thành các đoạn dài bằng một chu kỳ, nén chúng thành một số ít đoạn cơ sở, rồi học phụ thuộc thưa và động giữa các biến trực tiếp trên các đoạn cơ sở này bằng một nhánh trộn kênh dùng DropFusion, có đầu ra khởi tạo bằng không để hiệu chỉnh các cơ sở trước khi dự báo theo đoạn. Mục tiêu là bổ sung khả năng mô hình hóa liên biến cho một bộ dự báo theo đoạn độc lập kênh siêu nhẹ, mà vẫn giữ chi phí tính toán thấp và không làm mô hình kém đi so với backbone lúc khởi tạo. Thực nghiệm trên các bộ dữ liệu chuẩn và dữ liệu phụ tải cho thấy nhánh trộn kênh cải thiện độ chính xác của backbone trên mọi bộ dữ liệu, và DyBAS-Net đạt độ chính xác cạnh tranh với các mô hình hiện đại trong khi chi phí suy luận cùng bậc với các mô hình tuyến tính và thấp hơn nhiều lần so với các mô hình học sâu. Ưu điểm của mô hình là vừa nhẹ vừa tận dụng được thông tin liên biến; hạn chế là mức cải thiện so với backbone còn khiêm tốn khi phụ thuộc giữa các biến yếu, và mức độ thưa của DropFusion cần được chọn riêng cho từng bộ dữ liệu.**

***Keywords*—Dynamic dependency, Sparse segment, Multivariate time series, Load forecasting, Pattern extraction**

I. Introduction

A. General Introduction

Dự báo phụ tải được xem là năng lực nền tảng cho quy hoạch và vận hành hệ thống điện [1], và các hệ thống được quan sát thường là đa biến: một máy biến áp ghi nhận phụ tải cùng với nhiệt độ dầu, một công ty điện lực ghi nhận cùng lúc lượng tiêu thụ của hàng trăm khách hàng. Vì vậy, dự báo chuỗi thời gian đa biến (MTSF) phải dự báo đồng thời tất cả các biến, và việc nắm bắt phụ thuộc giữa các kênh đồng thời xử lý sự dịch chuyển phân phối ngày càng khó khăn khi quy mô và lĩnh vực ứng dụng mở rộng [2].

Hình 1 minh họa hai tính chất của loại dữ liệu này trên phụ tải điện Hà Nội. Thứ nhất là *sự lặp lại có chu kỳ*: các đoạn ngày gần như trùng hình dạng (Hình 1a, b), và khi xếp $L = 720$ giờ thành $N = 30$ đoạn ngày, mọi ngày có thể dựng lại từ chỉ 6 "ngày điển hình" với sai số 3,5% (Hình 1c). Tính hạng thấp này cho phép TimeBase dự báo chỉ với 0,39K tham số [3], và dữ liệu điện nói chung có một mẫu hình ngày dùng chung [4]. Thứ hai là *phụ thuộc giữa các biến thay đổi theo trạng thái*: tương quan giữa phụ tải sinh hoạt và nhiệt độ dương mạnh trong phần lớn thời gian nhưng đổi dấu, chủ yếu vào tháng 1 và tháng 3 (Hình 1d). Điều này khớp với nhận xét rằng cùng một cặp biến có thể liên kết mạnh ở chế độ này nhưng yếu ở chế độ khác [5], và tương quan trên từng mẫu có thể khác đáng kể tương quan toàn cục [6]. Vì vậy, mô hình độc lập kênh (CI) bền vững nhưng bỏ qua tương tác, còn mô hình phụ thuộc kênh (CD) nắm được tương tác nhưng nhạy với nhiễu từ các kênh không liên quan [7], [8].

Đồng thời, xu hướng dùng cửa sổ quá khứ rất dài (ví dụ $L = 720$) làm tăng số tham số và khối lượng tính toán của các bộ dự báo [3]. Do đó, một bộ dự báo phụ tải thực tiễn cần (i) đủ nhẹ để triển khai trên thiết bị hạn chế tài nguyên, (ii) khai thác được các mẫu hình chu kỳ, và (iii) học được phụ thuộc giữa các biến mà không khuếch đại các phụ thuộc giả. Mục tiếp theo cho thấy các phương pháp hiện có chỉ đáp ứng tối đa hai trong ba yêu cầu này.

B. Related Works

Các công trình liên quan được tổ chức theo năm từ khóa của bài báo. Với mỗi chủ đề, chúng tôi nêu bài toán mà chủ đề giải quyết, điểm qua các công trình tiêu biểu, so sánh chúng, chỉ ra ưu điểm, nhược điểm và khoảng trống mà DyBAS-Net hướng tới.

1) Sparse segment: Hướng nghiên cứu này thay việc mô hình hóa trục thời gian theo từng điểm bằng các phép toán trên đoạn (patch hoặc chu kỳ). PatchTST chia mỗi chuỗi đơn biến thành các patch mức chuỗi con, dùng làm token cho Transformer và chia sẻ mô hình giữa các kênh [9]. TimesNet biến đổi chuỗi 1D thành các tensor 2D theo các chu kỳ của nó, để biến thiên trong chu kỳ và giữa các chu kỳ được nắm bắt bằng tích chập 2D [10]. WITRAN truyền thông tin theo kiểu sóng nước ở hai mức độ chi tiết để nắm các mẫu lặp dài hạn và ngắn hạn, kèm mạng hồi tiếp tăng tốc cho dự báo tầm xa [11]. CFPT mô hình hóa tương tác liên tần số cùng với dấu thời gian nhận biết chu kỳ [12], và các mạng tương tác đa chu kỳ mô hình hóa tường minh tương tác giữa nhiều thành phần chu kỳ [13]; việc mô hình hóa tương tác tường minh như vậy được ghi nhận là cải thiện đáng kể độ chính xác [12],

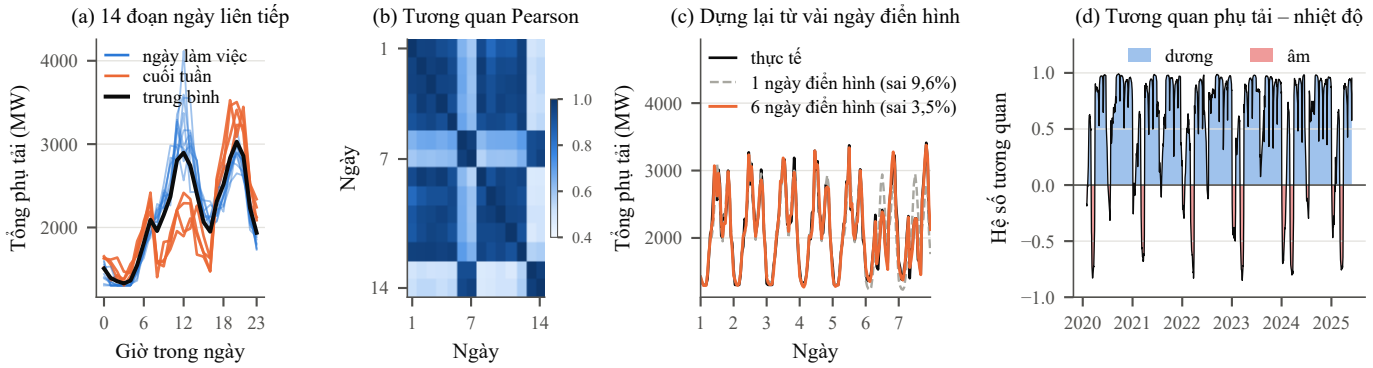


Fig. 1. Hai tính chất của dữ liệu phụ tải điện Hà Nội (theo giờ, 01/2020–05/2025). (a) 14 ngày liên tiếp của tổng phụ tải (từ 06/03/2023) chồng lên nhau. (b) Tương quan Pearson giữa 14 đoạn ngày đó; các ngày cuối tuần (6–7 và 13–14) tạo khối riêng, cho thấy thêm chu kỳ tuần. (c) Xếp $L = 720$ giờ thành ma trận $N \times P = 30 \times 24$ rồi biểu diễn mỗi ngày bằng tổ hợp tuyến tính của R "ngày điển hình" (xấp xỉ hạng R); hình vẽ 7 ngày đầu của cửa sổ, sai số tương đối trung bình trên mọi cửa sổ ghi trong chú giải. Một ngày điển hình bỏ sót cuối tuần, còn $R = 6$ gần như trùng số liệu thật. (d) Tương quan cuộn 30 ngày giữa phụ tải sinh hoạt và hệ số nhiệt độ, dao động từ $-0{,}85$ đến $0{,}99$.

[13]. So sánh với nhau, PatchTST dùng patch có độ dài cố định, còn TimesNet và các mô hình đa chu kỳ tổ chức chuỗi theo chu kỳ của dữ liệu. Ưu điểm của chúng là token mức đoạn bền vững hơn với nhiễu cục bộ và rút ngắn chuỗi cần xử lý. Nhược điểm là các đoạn được đưa vào các backbone sâu (Transformer, tích chập hoặc hồi tiếp), nên việc giảm độ dài chuỗi không dẫn đến một mô hình nhỏ. Trường hợp cực đoan của tính thưa mức đoạn, tức dự báo hoàn toàn tuyến tính giữa các chu kỳ (SparseTSF [14]) hoặc trên một số ít đoạn cơ sở (TimeBase [3]), cực kỳ nhẹ nhưng xử lý mỗi biến một cách độc lập. Khoảng trống là một biểu diễn mức đoạn gọn như TimeBase nhưng vẫn trao đổi được thông tin giữa các biến.

2) Multivariate time series: MTSF nhằm mô hình hóa đồng thời nhiều biến phụ thuộc lẫn nhau [2]. Các thiết kế tiêu biểu học biểu diễn tổng quát bằng học tương phản [15], tăng cường phân tích mùa vụ–xu hướng bằng nhiều wavelet [16], hợp nhất dữ liệu đa thang đo bằng mạng Kolmogorov–Arnold với hàm mất mát thích nghi [17], và kết hợp biểu diễn miền thời gian và miền tần số trong một Transformer [18]. Các Transformer trước đó cải thiện hiệu quả của attention trên chuỗi dài (Informer [19]) hoặc tích hợp phân rã với cấu trúc miền tần số (FEDformer [20]); JTFT còn dùng một lớp attention hạng thấp để nắm phụ thuộc giữa các chiều [18]. So sánh với nhau, các mô hình này khác nhau chủ yếu ở cách biểu diễn tín hiệu thời gian (embedding tương phản, wavelet, hợp nhất đa thang đo, token thời gian–tần số), nhưng cùng dựa trên một mạng sâu (Transformer, mạng wavelet, KAN). Ưu điểm của chúng là khả năng biểu diễn mạnh đối với động học phức tạp, phi tuyến. Nhược điểm, theo đánh giá của chúng tôi, là chi phí của backbone sâu lớn hơn nhiều so với một ánh xạ tuyến tính, và quan hệ liên biến được học lẫn trong backbone thay vì qua một cơ chế thưa, tường minh. Khoảng trống là một mô hình đa biến có thành phần liên biến thưa và thành phần thời gian rẻ như một ánh xạ tuyến tính.

3) Load forecasting: Dự báo phụ tải dự đoán nhu cầu điện hoặc năng lượng của tòa nhà, khách hàng và lưới điện. Dự báo phụ tải xác suất đã được hệ thống hóa trong các bài tổng quan dạng hướng dẫn, tóm tắt kỹ thuật, tiêu chuẩn đánh giá và những ngộ nhận phổ biến [1]. Các mô hình đa mùa vụ cổ điển vẫn được dùng rộng rãi: một mô hình ARIMA đa mùa vụ thích nghi dự báo phụ tải nước nóng sinh hoạt [21], và SARIMA kết hợp phân tích phổ các thành phần chu kỳ được dùng để dự báo nhu cầu điện đỉnh theo mùa [22]. Tuy vậy, các cách tiếp cận cổ điển như phân rã Fourier, làm trơn mũ và SARIMA chưa phản ánh đặc trưng riêng của từng chu kỳ mùa vụ cũng như tương tác giữa chúng [23]. Về phía học sâu, DeepAR học một mạng hồi tiếp (RNN) tự hồi quy toàn cục trên nhiều chuỗi liên quan và đưa ra toàn bộ phân phối dự báo [24], còn Temporal Fusion Transformer kết hợp cơ chế chọn biến với attention có thể diễn giải cho dự báo đa tầm với biến đồng hành tĩnh, đầu vào tương lai đã biết và tín hiệu lịch sử [25]. Các mô hình thống kê minh bạch và rẻ, nhưng dựa trên giả định tuyến tính cứng nhắc và thường xử lý các thành phần mùa vụ riêng rẽ; các mô hình sâu nắm bắt được tính phi tuyến và chia sẻ thông tin giữa các chuỗi, nhưng nặng và không mô hình hóa tường minh việc khách hàng hay cảm biến nào ảnh hưởng lẫn nhau. Khoảng trống là một bộ dự báo phụ tải giữ cấu trúc chu kỳ tường minh như các mô hình cổ điển, đồng thời học được các hiệu ứng phi tuyến và liên biến với chi phí nhỏ.

4) Dynamic dependency: Chủ đề này quan tâm đến cách học phụ thuộc giữa các biến khi chúng thưa, nhiễu và thay đổi theo thời gian. MS-FLOW xem tương tác liên biến là một dòng thông tin có dung lượng giới hạn: mô hình gộp các token patch thành trạng thái mức biến, chỉ giữ Top-K logit tương đồng cho mỗi biến trong một đồ thị thích nghi theo mẫu, và lan truyền thông tin dọc các cạnh này [5]. xCPD xem mỗi cặp kênh–patch là một nút đồ thị, chiếu các nút lên một cơ sở Fourier đồ thị dùng chung, nhóm chúng thành các dải tần thấp, trung và cao, rồi định tuyến mỗi patch tới các bộ lọc theo tần số bằng mixture of experts động; xCPD được dùng như một mô-đun cắm thêm trên backbone CI hoặc CD [26]. DUET phân cụm mềm các kênh trong miền tần số bằng metric Mahalanobis học được và mặt nạ lấy mẫu Bernoulli, rồi hợp nhất kết quả với

đặc trưng thời gian qua masked attention [8]. TQNet dùng các vector học được dịch chuyển theo chu kỳ làm query và đầu vào gốc làm key và value, để một lớp attention duy nhất kết hợp tương quan toàn cục và tương quan mức mẫu [6]. Cả bốn công trình đều thống nhất rằng việc trộn dày đặc, không giới hạn sẽ lan truyền nhiễu: MS-FLOW báo cáo biến thể dày đặc kém hơn định tuyến Top-K [5], và DUET báo cáo attention đầy đủ làm giảm độ chính xác trên ETTh2 [8]. Chúng khác nhau ở chỗ áp đặt tính thưa (cạnh Top-K, dải phổ, cụm mềm, query chu kỳ). Ưu điểm của chúng là mô hình hóa phụ thuộc bền vững, trong đó TQNet và xCPD còn chú trọng hiệu quả tính toán. Nhược điểm là phụ thuộc được học trên token mức patch hoặc mức điểm, nên đồ thị hoặc attention tăng theo số biến C (ma trận khoảng cách, xác suất và mặt nạ kích thước $C \times C$ giữa mọi cặp kênh trong DUET [8], đồ thị có nút là mọi cặp kênh–patch trong xCPD [26]); MS-FLOW cần chọn ngân sách thưa K theo từng bộ dữ liệu, dù không quá nhạy với K [5], và TQNet lưu ý rằng việc ép mô hình hóa đa biến mạnh có thể gây hại khi tương quan yếu [6]. Khoảng trống là học phụ thuộc thưa trên một biểu diễn nhỏ hơn nhiều so với chuỗi patch, được gắn vào sao cho không thể làm backbone kém đi.

5) Pattern extraction: Chủ đề này quan tâm đến cách trích xuất và tái sử dụng các mẫu hình lặp lại cho dự báo. CycleNet học các chu kỳ lặp $Q \in \mathbb{R}^{W \times C}$ với độ dài chu kỳ W dùng chung cho cả bộ dữ liệu, loại chúng khỏi đầu vào và chỉ dự báo phần dư; mô-đun này chỉ thêm $W \times C$ tham số và không thêm phép nhân–cộng nào [4]. TimeEmb tách thành phần bất biến theo thời gian, được truy xuất từ một ngân hàng embedding học được đánh chỉ số theo dấu thời gian, khỏi thành phần biến đổi theo thời gian được xử lý bằng bộ lọc tần số [27]. GTR duy trì một embedding thời gian toàn cục cho cả một chu kỳ, truy xuất đoạn khớp với vị trí tuyệt đối của đầu vào và hợp nhất nó với đầu vào qua tích chập 2D; mô-đun chỉ cần 40,1K tham số [28]. LightGTS cắt chuỗi thành các patch có độ dài bằng độ dài chu kỳ, để mỗi token bao phủ trọn một chu kỳ nội tại trên các bộ dữ liệu có thang thời gian khác nhau, và giải mã song song các giá trị tương lai [29]. So sánh với nhau, CycleNet, TimeEmb và GTR lưu các mẫu hình *toàn cục* học trên cả tập huấn luyện, còn LightGTS trích xuất các mẫu hình *cục bộ* căn theo chu kỳ từ cửa sổ đầu vào. Ưu điểm chung là cấu trúc chu kỳ được nắm bắt với chi phí thấp: CycleNet không thêm phép nhân–cộng [4], TimeEmb là một khung phân rã gọn nhẹ [27], và LightGTS nhỏ hơn 10–100 lần các mô hình nền tảng khác [29]. Nhược điểm chung là các mẫu hình được trích xuất theo từng kênh: CycleNet thua iTransformer trên Traffic và nêu việc mô hình hóa liên kênh là hướng phát triển tương lai [4], còn GTR cần thêm một mô-đun tổng hợp token toàn cục để cải thiện MSE trên Traffic 8,8–10,4% [28]. Khoảng trống là cho chính các mẫu hình đã trích xuất tương tác với nhau giữa các biến.

6) Research gap: Bảng I tổng hợp năm chủ đề. Các phương pháp dựa trên đoạn và trích xuất mẫu hình thì nhẹ và khai thác được tính chu kỳ, nhưng mô hình hóa mỗi biến độc lập hoặc dựa vào backbone nặng. Các phương pháp phụ thuộc động học được quan hệ liên biến thưa và thích nghi, nhưng trên token mức patch hoặc mức điểm có số lượng tăng theo đầu vào, nên không nhẹ bằng các mô hình tuyến tính mức đoạn. Thực tiễn dự báo phụ tải bị chia giữa các mô hình thống kê minh bạch nhưng tuyến tính và các mô hình sâu phi tuyến nhưng nặng. Theo hiểu biết của chúng tôi, chưa có mô hình nào *vừa* giữ được chi phí của một bộ dự báo mức đoạn siêu nhẹ *vừa* học được phụ thuộc liên biến thưa và động, cũng chưa có phương pháp nào học các phụ thuộc này trực tiếp trên các mẫu hình chu kỳ đã trích xuất (các đoạn cơ sở) theo cách giữ nguyên backbone CI tại thời điểm khởi tạo.

C. Research Objective

Bài báo đề xuất **DyBAS-Net** (**Dy**namic cross-variable dependency on **BAS**is segments) với các mục tiêu sau:

- *Trích xuất mẫu hình dựa trên đoạn thưa.* Phân đoạn mỗi chuỗi theo chu kỳ tự nhiên, chuẩn hóa từng pha và nén N đoạn quá khứ thành $R \ll N$ đoạn cơ sở trực giao.
- *Học phụ thuộc động trong không gian cơ sở.* Học phụ thuộc giữa các biến bằng CrossVBlock, DropFusion và IntraVBlock của DyTimeNet [31], áp dụng trên $P \cdot R$ hệ số cơ sở của mỗi biến thay vì L điểm gốc.
- *Hiệu chỉnh an toàn.* Đưa các phụ thuộc đã học vào dưới dạng một hiệu chỉnh cộng, khởi tạo bằng không, lên các cơ sở, để DyBAS-Net bắt đầu đúng từ backbone CI và chỉ bổ sung thông tin liên biến khi nó làm giảm hàm mất mát.
- *Hiệu quả.* Giữ phép tính trên trục thời gian ở các kích thước mức đoạn (N, R, P, cỡ hàng chục) thay vì L và S (cỡ hàng trăm), để chi phí suy luận gần với một mô hình tuyến tính theo đoạn.

D. Structure of the Paper

II. Methodology

A. Problem Formulation

Đầu vào. Đầu vào là một cửa sổ quá khứ $\mathcal{X} \in \mathbb{R}^{L \times C}$ độ dài L của C biến, $i = \overline{1, C}$; cửa sổ được trượt trên cả tập huấn luyện, kiểm định và kiểm tra:

$$\mathcal{X} = \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(C)} \\ x_2^{(1)} & x_2^{(2)} & \dots & x_2^{(C)} \\ \vdots & \vdots & \ddots & \vdots \\ x_L^{(1)} & x_L^{(2)} & \dots & x_L^{(C)} \end{bmatrix}, \tag{1}$$

trong đó $x_t^{(i)}$ là giá trị của biến thứ i tại thời điểm $t = \overline{1, L}$, đã chuẩn hóa Z-score bằng thống kê của tập huấn luyện. Giá trị thực của S bước tiếp theo là

$$\mathcal{Y} = \begin{bmatrix} x_{L+1}^{(1)} & x_{L+1}^{(2)} & \dots & x_{L+1}^{(C)} \\ \vdots & \vdots & \ddots & \vdots \\ x_{L+S}^{(1)} & x_{L+S}^{(2)} & \dots & x_{L+S}^{(C)} \end{bmatrix} \in \mathbb{R}^{S \times C}. \tag{2}$$

Đầu ra và mục tiêu. Giá trị dự báo $\hat{\mathcal{Y}} \in \mathbb{R}^{S \times C}$ của mọi biến được mô hình $\mathcal{F}$ (DyBAS-Net) ước lượng đồng thời, $\hat{\mathcal{Y}} = \mathcal{F}(\mathcal{X})$, với sai số

$$\mathcal{L}_{MSE} = \frac{1}{S \times C} \sum_{t=1}^{S} \sum_{i=1}^{C} \left(x_{L+t}^{(i)} - \hat{x}_{L+t}^{(i)} \right)^2. \tag{3}$$

TABLE I
SO SÁNH CÁC HƯỚNG TIẾP CẬN LIÊN QUAN THEO BA YÊU CẦU CỦA MỘT BỘ DỰ BÁO PHỤ TẢI ĐA BIẾN THỰC TIỄN.

Nhóm phương pháp	Công trình tiêu biểu	Biểu diễn chu kỳ / đoạn	Phụ thuộc liên biến	Gọn nhẹ
Đoạn thưa, tuyến tính	SparseTSF [14], TimeBase [3]	✓ chu kỳ, đoạn cơ sở	✗ (CI)	✓
Mô hình sâu theo đoạn	PatchTST [9], TimesNet [10], WITRAN [11]	✓ patch, chu kỳ 2D, mẫu lặp hai mức	✗ / ngầm định	✗
Mô hình sâu đa biến	Informer [19], FEDformer [20], JTFT [18], iTransformer [30]	✗ điểm, token tần số	✓ dày đặc / hạng thấp (JTFT)	✗
Mô hình phụ tải thống kê	ARIMA đa mùa vụ [21], SARIMA kết hợp phân tích phổ [22]	✓ thành phần mùa vụ	✗	✓ (tuyến tính)
Mô hình phụ tải học sâu	DeepAR [24], TFT [25]	✗	chỉ qua biến đồng hành	✗
Trích xuất mẫu hình	CycleNet [4], TimeEmb [27], GTR [28], LightGTS [29]	✓ chu kỳ toàn cục, token chu kỳ	✗ (CI)	✓
Phụ thuộc thưa / thích nghi	MS-FLOW [5], xCPD [26], DUET [8]	patch, cặp kênh–patch	✓ thưa, thích nghi	một phần
Attention với query chu kỳ	TQNet [6]	✓ query chu kỳ	✓ attention	một phần
DyBAS-Net (đề xuất)	–	✓ đoạn cơ sở	✓ thưa, động, trên cơ sở	✓

TABLE II
CÁC KÍCH THƯỚC DÙNG TRONG DYBAS-NET, KÈM GIÁ TRỊ CHO ETTH1 ($L = 720$, $S = 96$).

Ký hiệu	Ý nghĩa	ETTh1
P	Độ dài đoạn (chu kỳ)	24 (một ngày)
$N = \lceil L/P \rceil$	Số đoạn quá khứ	30 ngày
$N' = \lceil S/P \rceil$	Số đoạn tương lai	4 ngày
$R \ll N$	Số thành phần cơ sở	6
D	Độ rộng ẩn của nhánh trộn	64
$C' = \lfloor 0.75\, C \rfloor$	Độ rộng nút thắt của CrossVBlock	5
$D_{ff} = 2D$	Độ rộng ẩn của FFN trong IntraVBlock	128

Khi huấn luyện, một ràng buộc trực giao trên các thành phần cơ sở (Mục II-C) được cộng thêm:

$$\mathcal{L} = \mathcal{L}_{MSE} + \lambda_{orth}\, \mathcal{L}_{orth}, \qquad \lambda_{orth} \geq 0. \tag{4}$$

Kích thước và ký hiệu. Bảng II liệt kê các kích thước mô hình sử dụng. Chữ thường (x) ký hiệu vô hướng, chữ thường in đậm ($\mathbf{x}$) ký hiệu vector cột, chữ hoa in đậm ($\mathbf{X}$) ký hiệu ma trận, và chữ calligraphic ký hiệu dữ liệu hoặc đặc trưng ẩn của *tất cả* các biến ($\mathcal{X}$, $\mathcal{Y}$, $\mathcal{X}_{enc}$, …); riêng $\mathcal{M}$ ký hiệu mặt nạ và $\mathcal{L}$ ký hiệu hàm mất mát, còn trọng số và các ma trận trung gian ($\bar{\mathbf{E}}$, $\mathbf{H}$) vẫn in đậm. Chỉ số trên $^{(i)}$ chỉ biến thứ i, $[\cdot]_{p,n}$ chỉ phần tử ở hàng p cột n, chỉ số dưới của một vector in đậm chỉ một cột của ma trận tương ứng (ví dụ $\mathbf{e}_r^{(i)}$ là cột r của $\mathbf{E}^{(i)}$), và chỉ số dưới $_b$ chỉ mẫu thứ b trong một lô. Dấu ngã, dấu mũ và dấu phẩy trên lần lượt chỉ đại lượng đã chuẩn hóa, đại lượng dự báo và đại lượng đã hiệu chỉnh.

Tổng quan. DyBAS-Net xử lý một mẫu qua tám bước (Hình 2): (1) phân đoạn theo chu kỳ, cắt chuỗi độ dài L thành N đoạn độ dài P; (2) chuẩn hóa theo pha, trừ trung bình của từng pha trong chu kỳ; (3) trích xuất cơ sở, nén N đoạn thành R thành phần cơ sở $\mathbf{E}^{(i)}$; (4) mã hóa các hệ số cơ sở của mỗi biến thành một vector D chiều; (5) CrossVBlock với DropFusion, trộn thông tin giữa các biến; (6) IntraVBlock, biến đổi phi tuyến bên trong mỗi biến; (7) chiếu ngược, tạo hiệu chỉnh $\Delta\mathbf{E}^{(i)}$ cộng vào các cơ sở; và (8) dự báo đoạn và tái tạo chuỗi, sinh N' đoạn tương lai và cộng lại trung bình pha. Bước 1–2 tạo thành biểu diễn đoạn thưa (Mục II-B), bước 3 là trích xuất mẫu hình (Mục II-C), bước 4–7 là học phụ thuộc động (Mục II-D), và bước 8 hoàn thiện mô hình (Mục II-E). Bước 1–3 và 8 xử lý từng biến độc lập; chỉ bước 4–7 trao đổi thông tin giữa các biến.

B. Sparse Segment

Theo cách nhìn dựa trên chu kỳ của SparseTSF [14] và TimeBase [3], mỗi chuỗi được xử lý như một dãy các đoạn dài một chu kỳ thay vì từng bước thời gian riêng lẻ.

Bước 1: Phân đoạn theo chu kỳ. Chuỗi của biến i, $\mathbf{x}^{(i)} = (x_1^{(i)}, \ldots, x_L^{(i)})^\top$, được sắp xếp thành *ma trận đoạn* $\mathbf{X}^{(i)} \in \mathbb{R}^{P \times N}$ có các cột là các đoạn độ dài P:

$$\mathbf{X}^{(i)} = \begin{bmatrix} x_1^{(i)} & x_{P+1}^{(i)} & \cdots & x_{(N-1)P+1}^{(i)} \\ x_2^{(i)} & x_{P+2}^{(i)} & \cdots & x_{(N-1)P+2}^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ x_P^{(i)} & x_{2P}^{(i)} & \cdots & x_{NP}^{(i)} \end{bmatrix}, \tag{5}$$

tức $[\mathbf{X}^{(i)}]_{p,n} = x_{(n-1)P+p}^{(i)}$. Với ETTh1, $L = 720$ và $P = 24$ cho $N = 30$ cột, mỗi cột là một ngày, và $[\mathbf{X}^{(i)}]_{p,n}$ là giá trị tại giờ thứ p của ngày thứ n: $(n-1)$ ngày trọn vẹn trước ngày n chứa $(n-1)P$ điểm, đi thêm p bước nữa ta đến vị trí $(n-1)P + p$ của chuỗi gốc. Khi L không chia hết cho P, chuỗi được đệm ở đầu bằng $NP - L$ giá trị cuối của chu kỳ đầu tiên.

Bước 2: Chuẩn hóa theo pha. Trung bình của mỗi hàng (pha) trên N đoạn là

$$\mu_p^{(i)} = \frac{1}{N} \sum_{n=1}^{N} \left[\mathbf{X}^{(i)}\right]_{p,n}, \qquad \boldsymbol{\mu}^{(i)} = \left(\mu_1^{(i)}, \ldots, \mu_P^{(i)}\right)^\top \in \mathbb{R}^P, \tag{6}$$

và được trừ khỏi mọi phần tử của hàng tương ứng:

$$\left[\tilde{\mathbf{X}}^{(i)}\right]_{p,n} = \left[\mathbf{X}^{(i)}\right]_{p,n} - \mu_p^{(i)}. \tag{7}$$

$\boldsymbol{\mu}^{(i)}$ là "đoạn trung bình" của biến i (ví dụ phụ tải ban ngày cao hơn ban đêm); các bước sau chỉ học phần dao động quanh nó, và $\boldsymbol{\mu}^{(i)}$ được cộng lại ở bước 8. Từ đây, mọi phép toán trên trục thời gian tác động lên N đoạn thay vì L điểm, và đó là nguồn gốc của tính thưa: với $L = 720$ và $P = 24$, trục thời gian co từ 720 điểm còn 30 đoạn.

C. Pattern Extraction

Bước 3: Trích xuất cơ sở. Một ánh xạ tuyến tính *dùng chung cho mọi biến*, với tham số học được $\mathbf{W}_B \in \mathbb{R}^{N \times R}$ và $\mathbf{b}_B \in \mathbb{R}^R$, nén N đoạn thành R thành phần cơ sở [3]:

$$\mathbf{E}^{(i)} = \tilde{\mathbf{X}}^{(i)}\, \mathbf{W}_B + \mathbf{b}_B \in \mathbb{R}^{P \times R}, \tag{8}$$

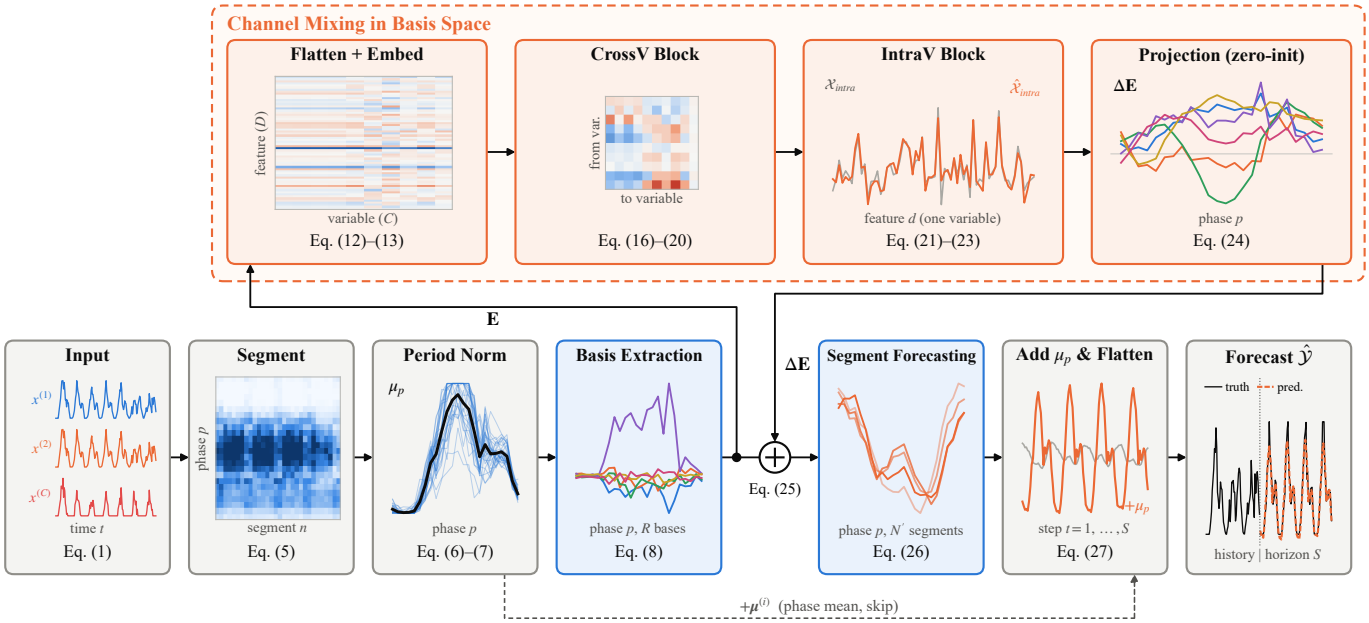


Fig. 2. Sơ đồ tổng quan của DyBAS-Net. Mỗi biến được phân đoạn theo chu kỳ, chuẩn hóa theo pha và nén thành R thành phần cơ sở $\mathbf{E}$ (nhánh độc lập kênh). Nhánh trộn kênh học phụ thuộc liên biến trên các cơ sở đã làm phẳng và trả về hiệu chỉnh $\Delta\mathbf{E}$ khởi tạo bằng không; các cơ sở đã hiệu chỉnh $\mathbf{E}' = \mathbf{E} + \Delta\mathbf{E}$ được ánh xạ thành các đoạn tương lai, khử chuẩn hóa và làm phẳng.

trong đó $\mathbf{b}_B$ được cộng vào mọi hàng. Cột r của $\mathbf{E}^{(i)}$ là một tổ hợp tuyến tính của N đoạn quá khứ $\tilde{\mathbf{x}}_n^{(i)}$ (các cột của $\tilde{\mathbf{X}}^{(i)}$), tức một "đoạn đại diện":

$$\mathbf{e}_r^{(i)} = \sum_{n=1}^{N} \left[\mathbf{W}_B\right]_{n,r} \tilde{\mathbf{x}}_n^{(i)} + \left[\mathbf{b}_B\right]_r \in \mathbb{R}^P. \tag{9}$$

Dựa trên tính hạng thấp của chuỗi dài [3], $N = 30$ ngày được tóm tắt bằng $R = 6$ "ngày điển hình".

Ràng buộc trực giao. Ma trận Gram tập hợp tích vô hướng giữa từng cặp thành phần cơ sở,

$$\mathbf{G}^{(i)} = \mathbf{E}^{(i)\top}\mathbf{E}^{(i)} \in \mathbb{R}^{R\times R}, \qquad \left[\mathbf{G}^{(i)}\right]_{r,s} = \left\langle \mathbf{e}_r^{(i)}, \mathbf{e}_s^{(i)} \right\rangle, \tag{10}$$

và phần ngoài đường chéo của nó bị phạt, lấy trung bình trên B mẫu của một lô và C biến:

$$\mathcal{L}_{orth} = \frac{1}{B\,C} \sum_{b=1}^{B} \sum_{i=1}^{C} \left\| \mathbf{G}_b^{(i)} - \operatorname{diag}\left(\mathbf{G}_b^{(i)}\right) \right\|_F. \tag{11}$$

$\mathcal{L}_{orth} = 0$ khi các thành phần cơ sở đôi một trực giao, nên mỗi thành phần mang một kiểu biến thiên riêng, không dư thừa. Số hạng này chỉ dùng khi huấn luyện; khi đánh giá chỉ dùng MSE và MAE. Ma trận cơ sở $\mathbf{E}^{(i)}$ chỉ gồm $P \cdot R$ số (ví dụ $24 \times 6 = 144$ thay vì 720), và DyBAS-Net học phụ thuộc liên biến chính trên các mẫu hình đã trích xuất này.

D. Dynamic Dependency

Bước 4–7 tạo thành nhánh trộn kênh. CrossVBlock, DropFusion và IntraVBlock được lấy từ DyTimeNet [31]; phép mã hóa ở bước 4 và phép chiếu ngược ở bước 7 là phần mới, cho phép nhánh này hoạt động trên các hệ số cơ sở thay vì các điểm thời gian gốc.

Bước 4: Mã hóa cơ sở. Mỗi $\mathbf{E}^{(i)}$ được làm phẳng theo hàng thành một vector, và C biến được xếp thành ma trận $\bar{\mathbf{E}} \in \mathbb{R}^{PR\times C}$:

$$\mathbf{e}^{(i)} = \operatorname{vec}\left(\mathbf{E}^{(i)}\right) \in \mathbb{R}^{PR}, \quad \bar{\mathbf{E}} = \left[\mathbf{e}^{(1)}, \ldots, \mathbf{e}^{(C)}\right]. \tag{12}$$

Một phép chiếu tuyến tính với tham số học được $\mathbf{W}_{enc} \in \mathbb{R}^{PR\times D}$ và bias $\mathbf{b}_{enc} \in \mathbb{R}^D$ (cộng vào mọi cột) ánh xạ chúng sang không gian ẩn D chiều:

$$\mathcal{X}_{enc} = \mathbf{W}_{enc}^{\top}\bar{\mathbf{E}} + \mathbf{b}_{enc} \in \mathbb{R}^{D\times C}. \tag{13}$$

Khác với lớp embedding của DyTimeNet vốn chiếu L điểm gốc của mỗi biến [31], đầu vào ở đây gồm PR hệ số cơ sở.

LayerNorm và GELU. LayerNorm [32] chuẩn hóa D đặc trưng của mỗi biến (một cột $\mathbf{h} \in \mathbb{R}^D$), đưa mọi biến về cùng thang đo trước khi trộn:

$$\mathrm{LN}(\mathbf{h}) = \boldsymbol{\gamma} \odot \frac{\mathbf{h} - \mu_{\mathbf{h}}}{\sqrt{\sigma_{\mathbf{h}}^2 + \varepsilon}} + \boldsymbol{\beta}, \tag{14}$$

trong đó $\mu_{\mathbf{h}}$ và $\sigma_{\mathbf{h}}^2$ là trung bình và phương sai các phần tử của $\mathbf{h}$, còn $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^D$ là tham số học được. Hàm kích hoạt là GELU [33],

$$\mathrm{GELU}(x) = x \cdot \Phi(x) = \frac{x}{2}\left(1 + \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right)\right), \tag{15}$$

với Φ là hàm phân phối tích lũy của $\mathcal{N}(0,1)$ và $\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}}\int_0^x e^{-t^2}dt$.

Bước 5: Khối liên biến (CrossVBlock). Sau chuẩn hóa, $\tilde{\mathcal{X}}_{enc} = \mathrm{LN}(\mathcal{X}_{enc}) \in \mathbb{R}^{D\times C}$, một MLP nút thắt dọc trục biến

$C \to C' \to C$ với $\mathbf{W}_{df,1} \in \mathbb{R}^{C \times C'}$ và $\mathbf{W}_{df,2} \in \mathbb{R}^{C' \times C}$ (bias $\mathbf{b}_1$, $\mathbf{b}_2$ cộng vào mọi hàng) tính, khi kiểm tra,

$$\mathbf{H} = \mathrm{GELU}\left(\tilde{\mathcal{X}}_{enc}\,\mathbf{W}_{df,1} + \mathbf{b}_1\right) \in \mathbb{R}^{D \times C'}, \tag{16}$$

$$\mathcal{X}_{cross} = \mathbf{H}\,\mathbf{W}_{df,2} + \mathbf{b}_2 \in \mathbb{R}^{D \times C}. \tag{17}$$

Mỗi *hàng* của $\tilde{\mathcal{X}}_{enc}$ chứa một đặc trưng của cả C biến, nên phép nhân phải với $\mathbf{W}_{df}$ tạo mỗi biến mới là tổ hợp của mọi biến trên đặc trưng đó, tức trộn thông tin *giữa* các biến. Khi huấn luyện, hai lớp tuyến tính này được thay bằng các lớp DropFusion [31].

DropFusion: mặt nạ cạnh $\mathcal{M}_1$ và mặt nạ nút $\mathcal{M}_2$. Xét lớp thứ nhất (đầu vào $\tilde{\mathcal{X}}_{enc} \in \mathbb{R}^{D \times C}$, trọng số $\mathbf{W}_{df,1} \in \mathbb{R}^{C \times C'}$). Ở giai đoạn thứ nhất, một *mặt nạ cạnh* $\mathcal{M}_1 \in \{0,1\}^{C \times C'}$, với mỗi phần tử bằng 0 với xác suất p_1, xóa kết nối giữa một biến đầu vào và một nút ẩn; mặt nạ được lấy mẫu lại cho mỗi lô:

$$\mathbf{W}_{masked} = \mathbf{W}_{df,1} \odot \mathcal{M}_1. \tag{18}$$

Ở giai đoạn thứ hai, một *mặt nạ nút* $\mathcal{M}_2 \in \{0,1\}^{D \times C'}$, với mỗi phần tử bằng 0 với xác suất p_2, tắt ngẫu nhiên từng đầu ra; mặt nạ được lấy mẫu riêng cho từng mẫu:

$$\mathbf{Y}_{masked} = \left(\tilde{\mathcal{X}}_{enc}\,\mathbf{W}_{masked} + \mathbf{b}_1\right) \odot \mathcal{M}_2. \tag{19}$$

Lớp thứ hai được xử lý tương tự, với đầu vào $\mathbf{H}$ và trọng số $\mathbf{W}_{df,2}$.

DropFusion: chuẩn hóa kỳ vọng. Khi huấn luyện, một phần kết nối bị tắt theo p_1 và p_2, còn khi kiểm tra mọi kết nối đều hoạt động, nên tổng tín hiệu sẽ tăng lên và làm lệch đầu ra. Các hệ số bù giữ kỳ vọng tín hiệu như nhau ở cả hai giai đoạn, cho ta lớp DropFusion hoàn chỉnh dùng khi huấn luyện:

$$\mathrm{DF}(\tilde{\mathcal{X}}_{enc}) = \frac{1}{1-p_2}\Big[\frac{\tilde{\mathcal{X}}_{enc}\,(\mathbf{W}_{df,1} \odot \mathcal{M}_1)}{1-p_1} + \mathbf{b}_1\Big] \odot \mathcal{M}_2. \tag{20}$$

Khi kiểm tra, $\mathcal{M}_1 = \mathcal{M}_2 = \mathbf{1}$, không áp dụng hệ số bù, và lớp này trở về lớp tuyến tính của (16)–(17).

Phụ thuộc học được là *thưa* vì mỗi bước huấn luyện chỉ thấy một tập con ngẫu nhiên các cạnh giữa các biến, ngăn mô hình dựa vào bất kỳ tương quan đơn lẻ nào, vốn có thể là tương quan giả. Phụ thuộc là *động* vì đồ thị phụ thuộc thưa thay đổi ở mỗi bước và các đặc trưng được trộn được tính từ các thành phần cơ sở của cửa sổ hiện tại, nên ảnh hưởng của một biến lên biến khác thay đổi theo từng mẫu.

Bước 6: Khối nội biến (IntraVBlock). Một kết nối tắt cộng đặc trưng liên biến vào đặc trưng đã chuẩn hóa để giữ thông tin gốc của mỗi biến:

$$\mathcal{X}_{intra} = \mathcal{X}_{cross} + \tilde{\mathcal{X}}_{enc} \in \mathbb{R}^{D \times C}. \tag{21}$$

Một mạng truyền thẳng dùng chung cho mọi biến được áp dụng lên *từng cột* (biến) $\mathbf{h} \in \mathbb{R}^D$,

$$\mathrm{FFN}(\mathbf{h}) = \mathbf{W}_4\ \mathrm{GELU}(\mathbf{W}_3\mathbf{h} + \mathbf{b}_3) + \mathbf{b}_4, \tag{22}$$

với $\mathbf{W}_3 \in \mathbb{R}^{D_{ff} \times D}$ và $\mathbf{W}_4 \in \mathbb{R}^{D \times D_{ff}}$, có LayerNorm phía trước và kết nối tắt phía sau:

$$\hat{\mathcal{X}}_{intra} = \mathcal{X}_{intra} + \mathrm{FFN}\left(\mathrm{LN}(\mathcal{X}_{intra})\right) \in \mathbb{R}^{D \times C}. \tag{23}$$

Dropout được áp dụng bên trong FFN khi huấn luyện.

Bước 7: Chiếu ngược và hiệu chỉnh cơ sở. Biểu diễn ẩn được chiếu ngược về không gian cơ sở,

$$\Delta\bar{\mathbf{E}} = \mathbf{W}_{out}^{\top}\,\hat{\mathcal{X}}_{intra} + \mathbf{b}_{out} \in \mathbb{R}^{PR \times C}, \qquad \mathbf{W}_{out} \in \mathbb{R}^{D \times PR}, \tag{24}$$

cột i được định hình lại thành $\Delta\mathbf{E}^{(i)} \in \mathbb{R}^{P \times R}$, và hiệu chỉnh được cộng vào các cơ sở:

$$\mathbf{E}'^{(i)} = \mathbf{E}^{(i)} + \Delta\mathbf{E}^{(i)}. \tag{25}$$

$\mathbf{W}_{out}$ và $\mathbf{b}_{out}$ được khởi tạo bằng không, nên $\Delta\mathbf{E}^{(i)} = \mathbf{0}$ khi bắt đầu huấn luyện và DyBAS-Net trùng với backbone độc lập kênh của nó. Vì vậy nhánh trộn chỉ học một *hiệu chỉnh*: nó dùng thông tin của các biến khác để điều chỉnh các "ngày điển hình" của biến i, và không thể khiến mô hình xuất phát từ một điểm kém hơn backbone.

Bước 8: Dự báo đoạn và tái tạo chuỗi. Một ánh xạ tuyến tính dùng chung sinh N' đoạn tương lai từ R cơ sở đã hiệu chỉnh,

$$\hat{\mathbf{Y}}^{(i)} = \mathbf{E}'^{(i)}\,\mathbf{W}_F + \mathbf{b}_F \in \mathbb{R}^{P \times N'}, \qquad \mathbf{W}_F \in \mathbb{R}^{R \times N'}. \tag{26}$$

Trung bình pha được cộng lại vào mọi đoạn tương lai, và các cột được nối thành một chuỗi, giữ lại S bước đầu:

$$\hat{x}^{(i)}_{L+t} = \left[\hat{\mathbf{Y}}^{(i)}\right]_{p,n} + \mu^{(i)}_p, \qquad t = (n-1)P + p,\ \ t = \overline{1,S}. \tag{27}$$

Ghép tất cả các biến cho $\hat{\mathcal{Y}} \in \mathbb{R}^{S \times C}$. Bỏ bước 4–7 ($\mathbf{E}'^{(i)} = \mathbf{E}^{(i)}$) ta thu lại TimeBase [3].

Điều chỉnh trọng số và tối ưu. Các trọng số được cập nhật ngược hướng gradient của hàm mục tiêu (4); ở dạng cơ bản,

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha\,\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_k}, \tag{28}$$

với $\boldsymbol{\theta}$ gồm mọi trọng số $\mathbf{W}$ và bias $\mathbf{b}$ và k là bước cập nhật; trong thực tế dùng bộ tối ưu Adam [34]. Hai tốc độ học được sử dụng: $\alpha = \alpha_B$ cho backbone $\mathbf{W}_B, \mathbf{b}_B, \mathbf{W}_F, \mathbf{b}_F$, và $\alpha = \alpha_M$ cho nhánh trộn. Tại epoch τ, tốc độ học giữ nguyên trong bốn epoch đầu, sau đó nhân với $0{,}8$ mỗi epoch,

$$\alpha^{(\tau)} = \begin{cases} \alpha, & \tau \le 4, \\ \alpha \cdot 0.8^{\,\tau-4}, & \tau > 4. \end{cases} \tag{29}$$

Huấn luyện dừng sớm khi MSE trên tập kiểm định không cải thiện trong năm epoch, và giữ lại bộ trọng số có MSE kiểm định thấp nhất.

Độ phức tạp. Với mỗi mẫu, backbone ((8) và (26)) cần $CPR(N+N')$ phép nhân–cộng (MACs), còn nhánh trộn cần

$$\mathrm{MACs}_M = \underbrace{2CPRD}_{\text{mã hóa/chiếu}} + \underbrace{2DCC'}_{\text{CrossV}} + \underbrace{2CDD_{ff}}_{\text{IntraV}}. \tag{30}$$

Không có số hạng nào tỉ lệ với $L \cdot S$; nhánh trộn không phụ thuộc vào L hay S. Với Electricity ($C = 321$, $L = S = 720$, $P = 24$, $R = 6$, $D = 64$), backbone cần khoảng 2,8M MACs và nhánh trộn khoảng 21,0M, tổng cộng 23,8M, trong khi DLinear cần $2CLS \approx 333$M MACs.

E. Proposed Model: DyBAS-Net

III. Experiments

IV. Conclusion

References

[1] T. Hong and S. Fan, "Probabilistic electric load forecasting: A tutorial review," *International Journal of Forecasting*, vol. 32, no. 3, pp. 914–938, 2016.

[2] J. Kim, H. Kim, H. Kim, D. Lee, and S. Yoon, "A comprehensive survey of deep learning for time series forecasting: architectural diversity and open challenges," *Artificial Intelligence Review*, vol. 58, p. 216, 2025.

[3] Q. Huang, Z. Zhou, K. Yang, Z. Yi, X. Wang, and Y. Wang, "TimeBase: The power of minimalism in efficient long-term time series forecasting," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 26 227–26 246.

[4] S. Lin, W. Lin, X. Hu, W. Wu, R. Mo, and H. Zhong, "CycleNet: Enhancing time series forecasting through modeling periodic patterns," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[5] F. Zhang, S. Fan, and H. Wang, "What if we let forecasting forget? A sparse bottleneck for cross-variable dependencies," in *Proceedings of the 43rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026.

[6] S. Lin, H. Chen, H. Wu, C. Qiu, and W. Lin, "Temporal query network for efficient multivariate time series forecasting," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 37 797–37 814.

[7] L. Han, H.-J. Ye, and D.-C. Zhan, "The capacity and robustness trade-off: Revisiting the channel independent strategy for multivariate time series forecasting," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 11, pp. 7129–7142, 2024.

[8] X. Qiu, X. Wu, Y. Lin, C. Guo, J. Hu, and B. Yang, "DUET: Dual clustering enhanced multivariate time series forecasting," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2025, pp. 1185–1196.

[9] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *The Eleventh International Conference on Learning Representations*, 2023.

[10] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "TimesNet: Temporal 2D-variation modeling for general time series analysis," in *The Eleventh International Conference on Learning Representations*, 2023.

[11] Y. Jia, Y. Lin, X. Hao, Y. Lin, S. Guo, and H. Wan, "WITRAN: Water-wave information transmission and recurrent acceleration network for long-range time series forecasting," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 12 389–12 456.

[12] F. Kou, J. Wang, L. Shi, Y. Yao, Y. Li, S. Zhu, Z. Zhang, and J. Du, "CFPT: Empowering time series forecasting through cross-frequency interaction and periodic-aware timestamp modeling," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 31 634–31 647.

[13] Y. Xie, L. Dang, and B. Chen, "Multi-period interaction networks for time series forecasting," *Pattern Recognition Letters*, vol. 198, pp. 29–35, 2025.

[14] S. Lin, W. Lin, W. Wu, H. Chen, and J. Yang, "SparseTSF: Modeling long-term time series forecasting with 1k parameters," in *Proceedings of the 41st International Conference on Machine Learning*, 2024, pp. 30 211–30 226.

[15] J. Hu, Z. Hu, T. Li, and S. Du, "A contrastive learning based universal representation for time series forecasting," *Information Sciences*, vol. 635, pp. 86–98, 2023.

[16] Z. Zhang, J. Qin, D. Sun, Z. Zhang, J. Ma, X. Feng, and H. Zhang, "MWENet: Multi-wavelet enhanced network for seasonal-trend analysis in long-term forecasting," *Neurocomputing*, vol. 649, p. 130730, 2025.

[17] J. Liu, F. Yang, and K. Yan, "Multi-scale data fusion and adaptiveloss Kolmogorov–Arnold network for multivariate time series forecasting," *Information Fusion*, vol. 125, p. 103454, 2026.

[18] Y. Chen, S. Liu, J. Yang, H. Jing, W. Zhao, and G. Yang, "A joint time-frequency domain transformer for multivariate time series forecasting," *Neural Networks*, vol. 176, p. 106334, 2024.

[19] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 11 106–11 115.

[20] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 27 268–27 286.

[21] H. Nagpal, A. Soppelsa, H. Madsen, and R. Fedrizzi, "An adaptive multi-seasonal ARIMA approach for domestic hot water load forecasting: A pilot study," *Energy and Buildings*, vol. 347, p. 116226, 2025.

[22] E. A. Melchor Cruz, L. A. Hernández Armenta, and M. Badaoui, "Seasonal forecasting of peak electricity demand using spectral analysis," *Results in Engineering*, vol. 28, p. 107338, 2025.

[23] H. Moon, H. Lee, and B. Song, "Mixed pooling of seasonality for time series forecasting: An application to pallet transport data," *Expert Systems with Applications*, vol. 201, p. 117195, 2022.

[24] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks," *International Journal of Forecasting*, vol. 36, no. 3, pp. 1181–1191, 2020.

[25] B. Lim, S. Ö. Arık, N. Loeff, and T. Pfister, "Temporal fusion transformers for interpretable multi-horizon time series forecasting," *International Journal of Forecasting*, vol. 37, no. 4, pp. 1748–1764, 2021.

[26] D. Li, S. Zheng, C. Xu, J. Bian, and R. Jiang, "Routing channel-patch dependencies in time series forecasting with graph spectral decomposition," in *The Fourteenth International Conference on Learning Representations*, 2026.

[27] M. Xia, C. Zhang, Z. Zhang, H. Miao, Q. Liu, Y. Zhu, and B. Yang, "TimeEmb: A lightweight static-dynamic disentanglement framework for time series forecasting," in *Advances in Neural Information Processing Systems*, vol. 38, 2025.

[28] F. Cao, L. Dai, J. Han, and H. Xiong, "Enhancing multivariate time series forecasting with global temporal retrieval," in *The Fourteenth International Conference on Learning Representations*, 2026.

[29] Y. Wang, Y. Qiu, P. Chen, Y. Shu, Z. Rao, L. Pan, B. Yang, and C. Guo, "LightGTS: A lightweight general time series forecasting model," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 64 109–64 126.

[30] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long, "iTransformer: Inverted transformers are effective for time series forecasting," in *The Twelfth International Conference on Learning Representations*, 2024.

[31] T. Chen, J. Lai, Y. Sun, and W. K. V. Chan, "DyTimeNet: Dynamic cross-variable dependency network with sparse strategy for multivariate time series forecasting," *Neurocomputing*, vol. 676, p. 132964, 2026.

[32] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[33] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," *arXiv preprint arXiv:1606.08415*, 2016.

[34] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.